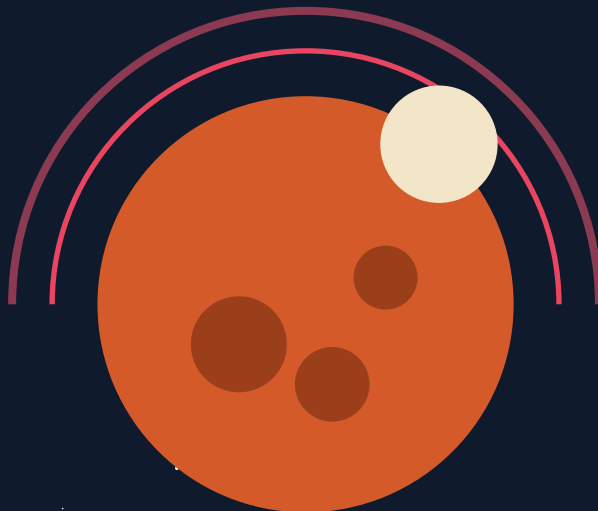




MISSÃO AURORA SIGER | MARTE

SIGER-3

Sistema Inteligente de Gestão de Energia

Colônia Aurora Siger · Hellas Planitia, Marte

Relatório Técnico – Fase 3 | Análise de Dados, Previsão e Decisão Automatizada

■ INFORMAÇÕES DO PROJETO

Colônia:	Aurora Siger — Hellas Planitia, Marte
Disciplina:	Estruturas de Dados e Inteligência Artificial
Sistema:	SIGER-3 (Sistema Integrado de Gestão de Energia – Fase 3)
Repositório:	github.com/VictorCamargo344/Aurora-Fase3
Ano:	2026

1. Sumário Executivo

Este relatório traz as informações sobre o SIGER-3, o sistema integrado de gestão de energia da Colônia Aurora Siger em Marte, mais especificamente em Hellas Planitia. O sistema foi desenvolvido em Python puro e se organiza em seis etapas, desde a estruturação dos dados até a simulação de cenários críticos. O foco do SIGER-3 é integrar a organização dos dados, uma lógica automatizada para tomada de decisão e previsões por regressão linear, tudo isso pra garantir a segurança energética e a continuidade das operações na colônia.

O projeto dá continuidade ao MGPEB (Módulo de Gerenciamento de Pouso e Estabilização de Base) da Fase 2, reaproveitando a frota de módulos que pousou na colônia e agora integra a operação contínua da base. Essa continuidade narrativa demonstra a evolução do sistema de reativo (autorização de pouso) para preditivo (análise inteligente).

Equipe: Lucas Araujo de Carvalho · Pietra Fanticelli · Rafael Gonçalves de Souza Pereira · Victor de Camargo Gomes

■ Repositório público: github.com/VictorCamargo344/Aurora-Fase3

2. Organização dos Dados da Colônia (Etapa 1)

Os dados foram estruturados numa arquitetura hierárquica em três camadas, usando dicionários aninhados, listas e tabelas chave-valor.

2.1 Estrutura Hierárquica Principal

O dicionário **dados_colonia** armazena todas as informações da colônia divididas em três subsistemas: energético, ambiental e operacional:

```
dados_colonia = {
    'nome': 'Aurora Siger',
    'localizacao': 'Marte – Hellas Planitia',
    'sistemas': {
        'energetico': {
            'solar': {'paineis': 120, 'geracao_atual_kwh': 45.0, 'eficiencia_pct': 78},
            'eolico': {'turbinas': 4, 'geracao_atual_kwh': 18.0, 'velocidade_vento_kmh':
12.0},
            'reserva': {'nivel_kwh': 320.0, 'capacidade_kwh': 500.0, 'percentual_pct': 64.0}
        },
        'ambiental': {
            'temperatura_interna_c': 22.0, 'temperatura_externa_c': -63.0,
            'velocidade_vento_kmh': 12.0, 'pressao_pa': 636.0
        },
        'operacional': { 'modulos': [ ... ] }
    }
}
```

2.2 Módulos Operacionais – Prioridade de Suporte à Vida

Os módulos estão organizados como uma lista de dicionários que incluem nome, tipo, consumo e prioridade (menor número = mais crítico). Esses são os mesmos módulos que pousaram com sucesso no MGPEB (Fase 2):

Módulo	Tipo	Consumo (kWh)	Prioridade
Habitat Alpha	Habitação	30,0	1 – Crítico
MedBase	Medicina	20,0	2 – Alta
Reator Solar	Energia	15,0	2 – Alta

BioLab	Laboratório	10,0	3 – Média
LogiStore	Logística	8,0	4 – Baixa
TerraBot	Mineração	12,0	5 – Opcional
TOTAL		95,0	

Tabela 1 – Módulos operacionais e consumo energético

2.3 Resumo de Sensores – Tabela Chave-Valor

A função `obter_resumo_sensores()` coleta os dados mais importantes do dicionário hierárquico e os retorna num formato chave-valor simples:

```
sensores = {
    'geracao_solar_kwh': 45.0,
    'geracao_eolica_kwh': 18.0,
    'geracao_total_kwh': 63.0,
    'reserva_pct': 64.0,
    'temperatura_interna_c': 22.0,
    'temperatura_externa_c': -63.0,
    'velocidade_vento_kmh': 12.0
}
```

■ Essa separação entre a estrutura hierárquica e o resumo chave-valor facilita a leitura e manutenção do código.

3. Cálculo de Geração e Consumo (Etapa 2)

O sistema faz cálculos dinâmicos para determinar a geração total e o consumo total com base nos dados estruturados:

```
def calcular_geracao_total(dados):
    sis = dados['sistemas']['energetico']
    return sis['solar']['geracao_atual_kwh'] + sis['eolico']['geracao_atual_kwh']

def calcular_consumo_total(dados):
    modulos = dados['sistemas']['operacional']['modulos']
    return sum(m['consumo_kwh'] for m in modulos if m['status'] == 'ativo')

# Estado base da colonia:
geracao = calcular_geracao_total(dados_colonia) # 45.0 + 18.0 = 63.0 kWh
consumo = calcular_consumo_total(dados_colonia) # soma dos 6 modulos = 95.0 kWh
reserva = dados_colonia['sistemas']['energetico']['reserva']['percentual_pct'] # 64.0%
```

4. Regras de Decisão Automatizada (Etapa 3)

O motor de decisão `tomar_decisao()` analisa cinco variáveis — geração, consumo, reserva, temperatura externa e velocidade do vento — para fornecer um diagnóstico completo com status e ações recomendadas. A função também imprime o relatório formatado e retorna os valores para uso posterior.

4.1 Tabela de Regras de Decisão

Status	Condição	Ações Recomendadas
CRÍTICO	Consumo > Geração E Reserva < 20%	Desligar TerraBot e LogiStore; manter apenas P1 e P2
ALERTA	Consumo > Geração (reserva ok)	Ativar modo economia; reduzir TerraBot e LogiStore

ATENÇÃO	Reserva < 50% E consumo > 80% da geração	Reduzir operação de BioLab e LogiStore
EFICIENTE	Geração > Consumo × 1,5	Armazenar energia excedente nas baterias
NORMAL	Nenhuma das anteriores	Manter operação atual — sistema estável

Tabela 2 – Regras de decisão do SIGER-3

4.2 Implementação – Motor de Decisão

```
def tomar_decisao(geracao, consumo, reserva_pct, temp_ext=0, vento=0):
    saldo = geracao - consumo
    alertas, acoes = [], []

    # Regra Critica
    if consumo > geracao and reserva_pct < 20:
        status = 'CRITICO'
        acoes.extend(['Desligar TerraBot imediatamente',
                     'Desligar LogiStore',
                     'Manter Habitat Alpha'])
    elif consumo > geracao:
        status = 'ALERTA'
    elif reserva_pct < 50 and consumo > geracao * 0.80:
        status = 'ATENCAO'
    elif geracao > consumo * 1.5:
        status = 'EFICIENTE'
    else:
        status = 'NORMAL'

    # Regras extras
    if temp_ext < -150: acoes.append('Verificar isolamento dos modulos')
    if vento > 80: acoes.append('Recolher paineis solares externos')
    return status, alertas, acoes
```

4.3 Priorizar Módulos – Bubble Sort

A função **priorizar_modulos()** organiza os módulos por prioridade utilizando bubble sort implementado manualmente:

```
def priorizar_modulos(modulos):
    lista = list(modulos)
    n = len(lista)
    for i in range(n):
        for j in range(0, n - i - 1):
            if lista[j]['prioridade'] > lista[j+1]['prioridade']:
                lista[j], lista[j+1] = lista[j+1], lista[j]
    return lista
```

5. Previsão por Regressão Linear (Etapa 4)

Esse módulo aplica regressão linear simples pra estimar a energia gerada pelas turbinas com base na velocidade do vento.

5.1 Dados Históricos

```
historico_vento_kmh = [4, 6, 8, 10, 12, 14, 16, 18, 20]
historico_energia_kwh = [10, 15, 20, 25, 30, 35, 40, 45, 50]
```

5.2 Cálculo dos Coeficientes – Mínimos Quadrados

```
def calcular_regressao_linear(x, y):
    n = len(x)
    soma_x = sum(x)
    soma_y = sum(y)
    soma_xy = sum(x[i] * y[i] for i in range(n))
    soma_x2 = sum(xi ** 2 for xi in x)

    m = (n * soma_xy - soma_x * soma_y) / (n * soma_x2 - soma_x ** 2)
    b = (soma_y - m * soma_x) / n
    return round(m, 4), round(b, 4)

# Resultado: m = 2.5, b = 0.0
# Formula: Energia = 2.5 * Vento + 0.0
```

■ **Exemplo do enunciado:** com vento de 11 km/h, a previsão de energia é **27,5 kWh**, atendendo exatamente ao requisito da atividade ("vento = 11 → energia ≈ 27").

5.3 Limitação Física do Modelo Linear

Vale destacar que, na física real, a potência eólica cresce com o cubo da velocidade do vento ($P = \frac{1}{2} \cdot \rho \cdot A \cdot v^3$), e o **Limite de Betz** restringe o aproveitamento máximo a 59,3% dessa potência. O modelo linear foi escolhido conforme orientação da disciplina, sendo adequado para a faixa operacional treinada. Para previsões fora dessa faixa, um modelo cúbico ajustado seria mais preciso. Para mitigar essa limitação, o sistema implementa limites operacionais que travam a saída em valores fisicamente coerentes.

5.4 Limites Operacionais da Turbina

O modelo abrange os limites físicos da turbina marciana, garantindo previsões realistas dentro das faixas seguras:

Faixa de Vento	Comportamento	Saída
< 10 km/h (cut-in)	Turbina parada — sem geração	0,0 kWh / PARADA
10 a 53 km/h (operação)	Regressão linear aplicada	$y = 2,5x + 0$
54 a 71 km/h (nominal)	Potência máxima constante	50 kWh / POT. MAX
≥ 72 km/h (cut-out)	Desligada por segurança	0,0 kWh / DESLIGADA

Tabela 3 – Limites operacionais da turbina eólica

5.5 Previsões de Referência

Vento (km/h)	Energia prevista (kWh)	Status
5	0,0	PARADA
11	27,5	OPERANDO (exemplo do enunciado)
25	62,5	OPERANDO
55	50,0	POTÊNCIA MÁXIMA
80	0,0	DESLIGADA – SEGURANÇA

Tabela 4 – Exemplos de previsão de energia eólica

6. Sistema Integrado – Ciclo SIGER-3 (Etapa 5)

A função **executar_ciclo_siger3()** integra todos os módulos num ciclo completo de processamento: leitura de sensores → previsão eólica → decisão automatizada → lista de módulos priorizados.

```
def executar_ciclo_siger3(nome_cenario, dados):  
    # --- ENTRADA ---  
    geracao = calcular_geracao_total(dados)  
    consumo = calcular_consumo_total(dados)  
    reserva = dados['sistemas']['energetico']['reserva']['percentual_pct']  
    vento = dados['sistemas']['ambiental']['velocidade_vento_kmh']  
    temp_ext = dados['sistemas']['ambiental']['temperatura_externa_c']  
  
    # --- PROCESSAMENTO: previsao eolica ---  
    prev_eolica, status_eolico = prever_energia_eolica(vento, m_reg, b_reg)  
  
    # --- DECISAO ---  
    tomar_decisao(geracao, consumo, reserva, temp_ext, vento)  
  
    # --- Modulos por prioridade ---  
    modulos_ord = priorizar_modulos(dados['sistemas']['operacional']['modulos'])
```

■ **copy.deepcopy()** é usado para criar cópias independentes dos dados de cada cenário, preservando o dicionário original sem efeitos colaterais entre simulações. Por isso, valores como geração e reserva podem variar entre os cenários da Tabela 5.

7. Simulações de Cenários (Etapa 6)

Validamos o SIGER-3 com seis cenários que vão desde operação eficiente até tempestades marcianas extremas. Cada cenário usa uma cópia ajustada do dicionário base (via copy.deepcopy) para alterar valores específicos sem afetar o estado original da colônia:

Cenário	Geração	Consumo	Reserva	Vento	Status
Operação Normal	110 kWh	95 kWh	64%	12 km/h	NORMAL
Alerta de Consumo	55 kWh	95 kWh	60%	12 km/h	ALERTA
Situação Crítica	35 kWh	95 kWh	12%	12 km/h	CRÍTICO
Operação Eficiente	180 kWh	95 kWh	85%	12 km/h	EFICIENTE
Tempestade Marciana	15 kWh	95 kWh	45%	95 km/h	CRÍTICO + extras
Modo Economia	63 kWh	75 kWh	64%	12 km/h	ALERTA/ATENÇÃO

Tabela 5 – Cenários de simulação e resultados esperados

■ No Cenário **Modo Economia**, os módulos TerraBot e LogiStore têm o status alterado para 'inativo', reduzindo o consumo total de 95 kWh para 75 kWh — demonstrando que o sistema aplica corretamente as ações recomendadas.

8. Como o Sistema Beneficia a Colônia Aurora Siger

- 1. Segurança na Vida:** Os módulos críticos nunca são desligados automaticamente, garantindo a sobrevivência dos colonos.
- 2. Resposta Automática:** O sistema consegue distinguir cinco níveis diferentes permitindo respostas proporcionais sem exageros.
- 3. Previsões Antecipadas:** O modelo prevê geração antes das leituras diretas possibilitando planejamentos proativos.
- 4. Gestão Inteligente:** Detecta quando há excesso na geração instruindo o armazenamento, maximizando o uso das baterias disponíveis.

5. Adaptação ao Ambiente Marciano: As regras extras demonstram que o sistema tá bem calibrado pro ambiente específico lá em Marte.

6. Continuidade da Missão: Aproveita a frota de módulos já pousada na Fase 2 (MGPEB), demonstrando evolução natural de um sistema reativo para um sistema preditivo.

9. Repositório GitHub

O código-fonte do Siger-3 está disponível publicamente no repositório abaixo:

<https://github.com/VictorCamargo344/Aurora-Fase3>

10. Considerações Finais

O Siger-3 mostra como conceitos básicos podem ser aplicados numa solução robusta para um ambiente tão desafiador como Marte. A organização hierárquica facilita a navegação enquanto o motor transforma dados brutos em ações claras. As validações cobrem toda a faixa operacional, garantindo confiabilidade antes do deploy real na colônia. O reconhecimento das limitações do modelo linear (frente à física real do vento) demonstra pensamento crítico sobre o domínio do problema — um passo fundamental para evoluir o sistema em fases futuras.